

Sieve

筛法

素数筛法

引入

如果我们想要知道小于等于 n 有多少个素数呢？

一个自然的想法是对于小于等于 n 的每个数进行一次质数检验。这种暴力的做法显然不能达到最优复杂度。

埃拉托斯特尼筛法

过程

考虑这样一件事情：对于任意一个大于 1 的正整数 p ，那么它的 p 倍就是合数（ 1 ）。利用这个结论，我们可以避免很多次不必要的检测。

如果我们从小到大考虑每个数，然后同时把当前这个数的所有（比自己大的）倍数记为合数，那么运行结束的时候没有被标记的数就是素数了。

实现



C++Python

```
1 vector<int> prime;
2 bool is_prime[N];
3
4 void Eratosthenes(int n) {
5     is_prime[0] = is_prime[1] = false;
6     for (int i = 2; i <= n; ++i) is_prime[i] = true;
7     for (int i = 2; i <= n; ++i) {
8         if (is_prime[i]) {
9             prime.push_back(i);
10            if ((long long)i * i > n) continue;
11            for (int j = i * i; j <= n; j += i)
12                // 因为从 2 到 i - 1 的倍数我们之前筛过了，这里直接从 i
13                // 的倍数开始，提高了运行速度
14                is_prime[j] = false; // 是 i 的倍数的均不是素数
15        }
16    }
17 }
```

```

1 prime = []
2 is_prime = [False] * N
3
4
5 def Eratosthenes(n):
6     is_prime[0] = is_prime[1] = False
7     for i in range(2, n + 1):
8         is_prime[i] = True
9     for i in range(2, n + 1):
10        if is_prime[i]:
11            prime.append(i)
12            if i * i > n:
13                continue
14            for j in range(i * i, n + 1, i):
15                is_prime[j] = False

```

以上为 **Eratosthenes 筛法**（埃拉托斯特尼筛法，简称埃氏筛法），时间复杂度是 。

▼ 证明

现在我们就来看看推导过程：

如果每一次对数组的操作花费 1 个单位时间，则时间复杂度为：

其中 p_i 表示第 i 小的素数， $\pi(n)$ 表示 n 的素数个数。 $\sum_{i=1}^n \pi(n/p_i)$ 表示第一层 for 循环，其中累加上界为 n/p_i 为 $\text{if } (\text{prime}[i])$ 进入 true 分支的次数； $\sum_{i=1}^n \pi(n/p_i^2)$ 表示第二层 for 循环的执行次数。

根据 Mertens 第二定理，存在常数 M 使得：

所以 **Eratosthenes 筛法** 的时间复杂度为 $O(n \log \log n)$ 。接下来我们证明 Mertens 第二定理的弱化版本：

根据 $\pi(x) \sim x / \log x$ ，可知第 n 个素数的大小为 $O(n \log n)$ 。于是就有

当然，上面的做法效率仍然不够高效，应用下面几种方法可以稍微提高算法的执行效率。

筛至平方根

显然，要找到直到 $\sqrt{n}$ 为止的所有素数，仅对不超过 $\sqrt{n}$ 的素数进行筛选就足够了。



C++Python

```

1 vector<int> prime;
2 bool is_prime[N];
3
4 void Eratosthenes(int n) {
5     is_prime[0] = is_prime[1] = false;
6     for (int i = 2; i <= n; ++i) is_prime[i] = true;
7     // i * i <= n 说明 i <= sqrt(n)
8     for (int i = 2; i * i <= n; ++i) {
9         if (is_prime[i])
10             for (int j = i * i; j <= n; j += i) is_prime[j] = false;
11     }
12     for (int i = 2; i <= n; ++i)
13         if (is_prime[i]) prime.push_back(i);
14 }

1 prime = []
2 is_prime = [False] * N
3
4
5 def Eratosthenes(n):
6     is_prime[0] = is_prime[1] = False
7     for i in range(2, n + 1):
8         is_prime[i] = True
9         # 让 i 循环到 <= sqrt(n)
10    for i in range(2, isqrt(n) + 1): # `isqrt` 是 Python 3.8 新增的函数
11        if is_prime[i]:
12            for j in range(i * i, n + 1, i):
13                is_prime[j] = False
14    for i in range(2, n + 1):
15        if is_prime[i]:
16            prime.append(i)

```

这种优化不会影响渐进时间复杂度，实际上重复以上证明，我们将得到，根据对数的性质，它们的渐进相同，但操作次数会明显减少。

只筛奇数

因为除 2 以外的偶数都是合数，所以我们可以直接跳过它们，只关心奇数就好。

首先，这样做能让我们内存需求减半；其次，所需的操作大约也减半。

减少内存的占用

我们注意到筛选时只需要 `bool` 类型的数组。`bool` 数组的一个元素一般占用 字节（即 比特），但是存储一个布尔值只需要 个比特就足够了。

我们可以使用 [位运算](#) 的相关知识，将每个布尔值压到一个比特位中，这样我们仅需使用 比特（即 字节）而非 字节，可以显著减少内存占用。这种方式被称为「位级压缩」。

值得一提的是，存在自动执行位级压缩的数据结构，如 C++ 中的 `vector<L_Tbool>` 和 `bitset<>`。

另外，`vector<L_Tbool>` 和 `bitset<>` 对程序有常数优化，时间复杂度 的埃氏筛在使用 `bitset<>` 或 `vector<L_Tbool>` 优化后，性能甚至超过时间复杂度 的欧拉筛。

参见 [bitset: 与埃氏筛结合](#)。

分块筛选

由优化「筛至平方根」可知，不需要一直保留整个 `is_prime[1...n]` 数组。为了进行筛选，只保留到 的素数就足够

了，即 `prime[1...sqrt(n)]`。并将整个范围分成块，每个块分别进行筛选。这样，我们就不必同时在内存中保留多个块，而且 CPU 可以更好地处理缓存。

设 S 是一个常数，它决定了块的大小，那么我们就有了 $\frac{n}{S}$ 个块，而块 k 包含了区间 $[kS, (k+1)S]$ 中的数字。我们可以依次处理块，也就是说，对于每个块 k ，我们将遍历所有质数（从 2 到 $\sqrt{n}$ ）并使用它们进行筛选。

值得注意的是，我们在处理第一个数字时需要稍微修改一下策略：首先，应保留 2 中的所有的质数；第二，数字 0 和 1 应该标记为非素数。在处理最后一个块时，不应该忘记最后一个数字 n 并不一定位于块的末尾。

以下实现使用块筛选来计算小于等于 n 的质数数量。

▼ 实现

```
1 int count_primes(int n) {
2     constexpr static int S = 10000;
3     vector<int> primes;
4     int nsqrt = sqrt(n);
5     vector<char> is_prime(nsqrt + 1, true);
6     for (int i = 2; i <= nsqrt; i++) {
7         if (is_prime[i]) {
8             primes.push_back(i);
9             for (int j = i * i; j <= nsqrt; j += i) is_prime[j] = false;
10        }
11    }
12    int result = 0;
13    vector<char> block(S);
14    for (int k = 0; k * S <= n; k++) {
15        fill(block.begin(), block.end(), true);
16        int start = k * S;
17        for (int p : primes) {
18            int start_idx = (start + p - 1) / p;
19            int j = max(start_idx, p) * p - start;
20            for (; j < S; j += p) block[j] = false;
21        }
22        if (k == 0) block[0] = block[1] = false;
23        for (int i = 0; i < S && start + i <= n; i++) {
24            if (block[i]) result++;
25        }
26    }
27    return result;
28 }
```

分块筛选的渐进时间复杂度与埃氏筛法是一样的（除非块非常小），但是所需的内存将缩小为 $\frac{n}{S}$ ，并且有更好的缓存结果。另一方面，对于每一对块和区间 $[kS, (k+1)S]$ 中的素数都要进行除法，而对于较小的块来说，这种情况要糟糕得多。因此，在选择常数 S 时要保持平衡。

块大小 S 取 $\sqrt{n}$ 到 $\sqrt[3]{n}$ 之间，可以获得最佳的速度。

线性筛法

埃氏筛法仍有优化空间，它会将一个合数重复多次标记。有没有什么办法省掉无意义的步骤呢？答案是肯定的。

如果能让每个合数都只被标记一次，那么时间复杂度就可以降到 $O(n)$ 了。

▼ 实现



C++Python

```

1 vector<int> pri;
2 bool not_prime[N];
3
4 void pre(int n) {
5     for (int i = 2; i <= n; ++i) {
6         if (!not_prime[i]) {
7             pri.push_back(i);
8         }
9         for (int pri_j : pri) {
10             if (i * pri_j > n) break;
11             not_prime[i * pri_j] = true;
12             if (i % pri_j == 0) {
13                 // i % pri_j == 0
14                 // 换言之, i 之前被 pri_j 筛过了
15                 // 由于 pri 里面质数是从小到大的, 所以 i 乘上其他的质数的结果一定会被
16                 // pri_j 的倍数筛掉, 就不需要在这里先筛一次, 所以这里直接 break
17                 // 掉就好了
18                 break;
19             }
20         }
21     }
22 }

```

```

1 pri = []
2 not_prime = [False] * N
3
4
5 def pre(n):
6     for i in range(2, n + 1):
7         if not not_prime[i]:
8             pri.append(i)
9             for pri_j in pri:
10                 if i * pri_j > n:
11                     break
12                 not_prime[i * pri_j] = True
13                 if i % pri_j == 0:
14                     """
15                     i % pri_j == 0
16                     换言之, i 之前被 pri_j 筛过了
17                     由于 pri 里面质数是从小到大的, 所以 i 乘上其他的质数的结果一定会被
18                     pri_j 的倍数筛掉, 就不需要在这里先筛一次, 所以这里直接 break
19                     掉就好了
20                     """
21                     break

```

上面的这种 **线性筛法** 也称为 **Euler 筛法**（欧拉筛法）。

▼ Note

注意到筛法求素数的同时也得到了每个数的最小质因子。

筛法求欧拉函数

注意到在线性筛中，每一个合数都是被最小的质因子筛掉。比如设 p 是 n 的最小质因子，那么线性筛的过程中 n 通过筛掉。

观察线性筛的过程，我们还需要处理两个部分，下面对 n 分情况讨论。

如果 n 是质数，那么 n 包含了 n 的所有质因子。

那如果 n 呢，这时 n 和 p 是互质的，根据欧拉函数性质，我们有：

实现



C++Python

```
1 vector<int> pri;
2 bool not_prime[N];
3 int phi[N];
4
5 void pre(int n) {
6     phi[1] = 1;
7     for (int i = 2; i <= n; i++) {
8         if (!not_prime[i]) {
9             pri.push_back(i);
10            phi[i] = i - 1;
11        }
12        for (int pri_j : pri) {
13            if (i * pri_j > n) break;
14            not_prime[i * pri_j] = true;
15            if (i % pri_j == 0) {
16                phi[i * pri_j] = phi[i] * pri_j;
17                break;
18            }
19            phi[i * pri_j] = phi[i] * phi[pri_j];
20        }
21    }
22 }
```

```
1 pri = []
2 not_prime = [False] * N
3 phi = [0] * N
4
5
6 def pre(n):
7     phi[1] = 1
8     for i in range(2, n + 1):
9         if not not_prime[i]:
10            pri.append(i)
11            phi[i] = i - 1
12            for pri_j in pri:
13                if i * pri_j > n:
14                    break
15                not_prime[i * pri_j] = True
16                if i % pri_j == 0:
17                    phi[i * pri_j] = phi[i] * pri_j
18                    break
19                phi[i * pri_j] = phi[i] * phi[pri_j]
```

筛法求莫比乌斯函数

定义

根据莫比乌斯函数的定义，设 n 是一个合数， p 是 n 的最小质因子， $n = p^k \cdot m$ ，有：

若是质数，有。

实现



C++Python

```
1 vector<int> pri;
2 bool not_prime[N];
3 int mu[N];
4
5 void pre(int n) {
6     mu[1] = 1;
7     for (int i = 2; i <= n; ++i) {
8         if (!not_prime[i]) {
9             mu[i] = -1;
10            pri.push_back(i);
11        }
12        for (int pri_j : pri) {
13            if (i * pri_j > n) break;
14            not_prime[i * pri_j] = true;
15            if (i % pri_j == 0) {
16                mu[i * pri_j] = 0;
17                break;
18            }
19            mu[i * pri_j] = -mu[i];
20        }
21    }
22 }
```

```
1 pri = []
2 not_prime = [False] * N
3 mu = [0] * N
4
5
6 def pre(n):
7     mu[1] = 1
8     for i in range(2, n + 1):
9         if not not_prime[i]:
10            pri.append(i)
11            mu[i] = -1
12            for pri_j in pri:
13                if i * pri_j > n:
14                    break
15                not_prime[i * pri_j] = True
16                if i % pri_j == 0:
17                    mu[i * pri_j] = 0
18                    break
19                mu[i * pri_j] = -mu[i]
```

筛法求约数个数

用 $d(n)$ 表示 n 的约数个数， $\alpha(n)$ 表示 n 的最小质因子出现次数。

约数个数定理

定理：若 $n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k}$ ，则

证明：我们知道 n 的约数有 $d(n)$ 共 $d(n)$ 个，根据乘法原理， n 的约数个数就是 $d(n)$ 。

实现

因为 是积性函数，所以可以使用线性筛。

在这里简单介绍一下线性筛实现原理。

1. 当 p 为质数时， $d[p] = 2$ ，同时设 $num[p] = 1$ ，其中 p 为 p 的最小质因子。
2. 当 p 为 i 的质因子时， $d[i * p] = d[i] / num[i * p] * (num[i * p] + 1)$ 。
3. 当 p 与 i 互质时， $d[i * p] = d[i] * 2$ 。



C++Python

```
1 vector<int> pri;
2 bool not_prime[N];
3 int d[N], num[N];
4
5 void pre(int n) {
6     d[1] = 1;
7     for (int i = 2; i <= n; ++i) {
8         if (!not_prime[i]) {
9             pri.push_back(i);
10            d[i] = 2;
11            num[i] = 1;
12        }
13        for (int pri_j : pri) {
14            if (i * pri_j > n) break;
15            not_prime[i * pri_j] = true;
16            if (i % pri_j == 0) {
17                num[i * pri_j] = num[i] + 1;
18                d[i * pri_j] = d[i] / num[i * pri_j] * (num[i * pri_j] + 1);
19                break;
20            }
21            num[i * pri_j] = 1;
22            d[i * pri_j] = d[i] * 2;
23        }
24    }
25 }
```

```
1 pri = []
2 not_prime = [False] * N
3 d = [0] * N
4 num = [0] * N
5
6
7 def pre(n):
8     d[1] = 1
9     for i in range(2, n + 1):
10         if not not_prime[i]:
11             pri.append(i)
12             d[i] = 2
13             num[i] = 1
14         for pri_j in pri:
15             if i * pri_j > n:
16                 break
17             not_prime[i * pri_j] = True
18             if i % pri_j == 0:
19                 num[i * pri_j] = num[i] + 1
20                 d[i * pri_j] = d[i] // num[i * pri_j] * (num[i * pri_j] + 1)
21                 break
22             num[i * pri_j] = 1
23             d[i * pri_j] = d[i] * 2
```


筛法求约数和

表示 的约数和， 表示 的最小质因子的。

实现



C++Python

```
1 vector<int> pri;
2 bool not_prime[N];
3 int g[N], f[N];
4
5 void pre(int n) {
6     g[1] = f[1] = 1;
7     for (int i = 2; i <= n; ++i) {
8         if (!not_prime[i]) {
9             pri.push_back(i);
10            g[i] = i + 1;
11            f[i] = i + 1;
12        }
13        for (int pri_j : pri) {
14            if (i * pri_j > n) break;
15            not_prime[i * pri_j] = true;
16            if (i % pri_j == 0) {
17                g[i * pri_j] = g[i] * pri_j + 1;
18                f[i * pri_j] = f[i] / g[i] * g[i * pri_j];
19                break;
20            }
21            f[i * pri_j] = f[i] * f[pri_j];
22            g[i * pri_j] = 1 + pri_j;
23        }
24    }
25 }
```

```
1 pri = []
2 not_prime = [False] * N
3 f = [0] * N
4 g = [0] * N
5
6
7 def pre(n):
8     g[1] = f[1] = 1
9     for i in range(2, n + 1):
10         if not not_prime[i]:
11             pri.append(i)
12             g[i] = i + 1
13             f[i] = i + 1
14         for pri_j in pri:
15             if i * pri_j > n:
16                 break
17             not_prime[i * pri_j] = True
18             if i % pri_j == 0:
19                 g[i * pri_j] = g[i] * pri_j + 1
20                 f[i * pri_j] = f[i] // g[i] * g[i * pri_j]
21                 break
22             f[i * pri_j] = f[i] * f[pri_j]
23             g[i * pri_j] = 1 + pri_j
```

一般的积性函数

假如一个 [积性函数](#) 满足：对于任意质数 p 和正整数 k ，可以在关于 k 的低次多项式时间内计算，那么可以在 $O(\sqrt{n})$ 时间内筛出 $\leq n$ 的值。

设合数 n 的质因子分解是 $n = p_1^{a_1} p_2^{a_2} \dots p_k^{a_k}$ ，其中 p_i 为质数，我们在线性筛中记录，假如 n 被筛掉（ p_i 是质数），那么 n 满足如下递推式：

假如 $n = p_i^k$ ，说明 n 就是某个质数的次幂，可以 $O(1)$ 计算；否则， $n = p_i^k m$ ，其中 m 与 p_i 互质，那么 n 满足如下递推式：

本节部分内容译自博文 [Решето Эратосфена](#) 与其英文翻译版 [Sieve of Eratosthenes](#)。其中俄文版版权协议为 **Public Domain + Leave a Link**；英文版版权协议为 **CC-BY-SA 4.0**。

本页面最近更新：2025/1/21 02:05:56，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [StudyingFather](#), [HeRaNO](#), [Mr-Python-in-China](#), [shuzhouliu](#), [TravorLZH](#), [weilycoder](#), [YOYO-UIAT](#), [1804040636](#), [383494](#), [aofall](#), [CCXXXI](#), [CoelacanthusHex](#), [countercurrent-time](#), [cubeheadsun](#), [Early0v0](#), [Enter-tainer](#), [frank-xjh](#), [gongyisheng](#), [Great-designer](#), [greyqz](#), [H-J-Granger](#), [hqztrue](#), [ImpleLee](#), [inkydragon](#), [lr1d](#), [LJFYC007](#), [luojiny1](#), [Lutra-Fs](#), [Marcythm](#), [MegaOwler](#), [Menci](#), [mgt](#), [NachtgeistW](#), [opsiff](#), [partychicken](#), [Peanut-Tang](#), [PerfectPan](#), [Persdre](#), [shawlleyw](#), [StableAgOH](#), [Steaunk](#), [SukkaW](#), [sunruisjtu2020](#), [TianKong-y](#), [Tiphereth-A](#), [TrisolarisHD](#), [untitledunrevised](#), [WAAutoMaton](#), [WineChord](#), [wkywkyQAQ](#), [wood3](#), [Xeonacid](#), [YanWQ-monad](#), [zhouyuyang2002](#), [代建杉](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用